

Building an AI Engine Optimization (AEO) System for Measuring How Well LLMs Answer Snowflake Developer Questions

Chanin Nantasenamat, Daniel Myers, Umesh Unnikrishnan

Developer Relations, Snowflake Inc.

Summary

AI coding assistants are now part of the Snowflake developer workflow, but there is no systematic way to measure whether those assistants give developers correct, current answers. We built an AEO benchmark that evaluates AI answer quality across 128 Snowflake developer questions spanning 32 product categories. Using a 2^4 factorial experiment design replicated across three respondent models (`claude-opus-4-6`, `claude-opus-4-7`, `openai-gpt-5.4`) with a five-model judge panel, we tested all 16 configurations of four augmentation factors (domain prompt, citation instruction, agentic tools, self-critique) for 48 total runs. For `claude-opus-4-6`, the best configuration (citation + agentic tools, no domain prompt) scored 75.9%, a 22.7 percentage-point (pp) improvement over the bare LLM baseline of 53.2%. Citation instruction is the dominant score factor (+9.5pp average), while self-critique was consistently counterproductive (−2.6pp average). These findings directly inform how Snowflake should configure its AI-powered developer tools. For product managers, the category-level analysis surfaces three documentation gap types across the 32 product categories: Implement is the weakest question type in 56% of categories (18 of 32), reflecting incomplete how-to tutorials and code examples; Debug is weakest in 31% of categories (10 of 32), pointing to missing troubleshooting guides and runbooks; and Explain is weakest in 2 of 32 categories, indicating that conceptual documentation is thin in some high-traffic product areas. These are not AI configuration problems. They are documentation coverage problems. The PM Action Framework in the Results section (Table 8) maps each gap type to a concrete documentation action and identifies the specific categories with the largest gaps.

1 Note on Audiences

This paper serves two distinct readers. **Engineering and platform teams** who configure AI developer tools will find the core value in the Introduction through the factorial experiment results: what levers actually improve answer quality, and by how much. **Product managers** responsible for specific Snowflake feature areas can skip directly to the Product Category Intelligence section (after the How Each Factor Affects Answer Quality subsection in Results): it contains per-category analysis of where AI currently struggles with developer questions about their product area, what that pattern reveals about documentation coverage gaps, and a concrete action framework tied to each gap type.

2 Introduction

In early 2026, Vercel built an AI Engine Optimization (AEO) system to track how AI coding agents reference and recommend their products. Their system measures brand visibility: does the agent mention Vercel products? Our system asks a different question: does the agent give the *right* answer to Snowflake developer questions?

When a developer asks an AI assistant “How do I create a Cortex Search Service?”, the quality of the answer depends on more than the model’s training data. What actually makes an AI assistant give better answers to developer questions?

- More instructions? Bigger system prompts?
- Access to tools and documentation?
- Self-review and revision loops?
- All of the above?

We built a controlled experiment to find out.

We designed a benchmark that goes beyond brand tracking to measure multi-dimensional answer quality (correctness, completeness, recency, citation, and recommendation) against expert-authored canonical answers. Rather than testing multiple competing agents, we tested all 16 augmentation configurations across three respondent models (`claude-opus-4-6`, `claude-opus-4-7`, and `openai-gpt-5.4`) for 48 total runs, isolating the effect of each deployment lever and whether that effect holds across models. The result is a data-backed framework for configuring AI developer tools on the Snowflake platform.

3 Methodology

3.1 Question Bank

We authored 128 questions across 32 Snowflake product categories, with exactly 4 questions per category. Categories span the full Snowflake developer surface (Table 7) including Cortex AI Functions, Cortex Search, Cortex Agents, Cortex Code, Dynamic Tables, Snowpark, Streamlit in Snowflake, Apache Iceberg Tables, Snowflake ML, Snowpark Container Services, Native Apps Framework, Data Pipelines (Streams, Tasks, Snowpipe), Data Governance & Security, Snowflake Fundamentals & Architecture, dbt Projects on Snowflake, Semantic Views & Cortex Analyst, Database Change Management, Snowflake Postgres, and others. Each question falls into one of four test types: Explain (32), Implement (32), Debug (32), or Compare (32). For every question, we wrote a canonical answer grounded in official Snowflake documentation and defined up to five must-have factual elements (up to 640 total binary pass/fail checks). Table 1 shows one representative question per test type to illustrate the breadth of cognitive demands placed on respondents.

Table 1. One question per test type (Explain, Implement, Debug, Compare) drawn from the 128-question bank, illustrating the spectrum of cognitive demands the benchmark places on respondents. The full bank spans 32 Snowflake product categories with exactly four questions per category.

Q#	Type	Question
Q9	Explain	What are Cortex Agents and how do they orchestrate across structured and unstructured data sources?
Q26	Implement	How do I create a Snowflake-managed Iceberg table with an external volume pointing to S3?
Q79	Debug	My Snowflake Postgres instance is running slowly. How do I run health checks (cache hit ratio, bloat, vacuum status, blocking queries) and interpret the results?
Q44	Compare	When should I use streams/tasks vs. Dynamic Tables for data transformation pipelines? What factors should drive the decision?

These four examples illustrate the spectrum of reasoning the benchmark demands. An Explain question tests conceptual understanding, an Implement question requires correct syntax and documented procedure, a Debug question requires diagnostic reasoning under a realistic failure scenario, and a Compare question demands nuanced knowledge of tradeoffs between similar options. The full 128-question bank follows the same four-type structure across all 32 categories.

3.2 Scoring Rubric

Each response was scored on five dimensions using a 0–2 scale (0 = miss, 1 = partial, 2 = full). Table 2 summarises what each dimension measures.

Table 2. Five scoring dimensions used by each judge, with the evaluative question each dimension addresses. Each dimension is scored 0 (miss), 1 (partial), or 2 (full), giving a maximum of 10 points per question.

Dimension	What It Measures
Correctness	Are the facts and code accurate per current Snowflake documentation?
Completeness	Does the response cover the full answer without omitting key steps or concepts?
Recency	Does it reflect current Snowflake features and syntax rather than deprecated approaches?
Citation	Does it reference or link to official Snowflake documentation?
Recommendation	Does it suggest the Snowflake-native path when one exists?

Maximum score per question: 10 points (5 dimensions × 2). Maximum per run: 1,280 points (128 questions × 10). The final score for each response is the panel average across five judges, expressed as a percentage of the maximum. Each question also has up to five must-have binary checks, producing a separate must-have (MH) pass rate.

3.3 Judge Panel

Every response was scored independently by five LLM judges: `claude-opus-4-6`, `claude-opus-4-7`, `openai-gpt-5.4`, `llama4-maverick`, and `gemini-3.1-pro`. The final score for each response is the panel average. This design mitigates single-model scoring bias and reduces the influence of any one model’s preferences or blind spots.

3.4 Scoring Pipeline: Custom vs TruLens Native

We built a custom scoring pipeline rather than using TruLens native feedback functions. This section explains the rationale and the enhancements our approach introduces.

TruLens provides a standard RAG Triad: groundedness (does the response stay faithful to retrieved context?), answer relevance (does the response address the question?), and context relevance (does the retrieved context address the question?). These three metrics are well-suited for general-purpose RAG evaluation but are not sufficient for a domain-specific developer benchmark. They do not measure factual correctness against a canonical answer, they do not check for the presence of specific must-have facts, and they do not capture dimensions like Recency (is current syntax used?) or Recommendation (does the response suggest the Snowflake-native approach?).

Our custom pipeline extends the TruLens baseline in four ways:

1. **Five-dimension rubric.** We score on Correctness, Completeness, Recency, Citation, and Recommendation using a 0–10 scale per dimension, producing a richer per-question profile than the binary RAG Triad metrics.
2. **Canonical answer grounding.** Each judge scores against an expert-authored canonical answer rather than against retrieved context alone. This catches cases where the retrieval is relevant but the response is still factually wrong or incomplete relative to the documented correct answer.
3. **Must-have binary checks.** In addition to rubric scores, each question has up to five must-have factual elements that produce a separate pass/fail signal. This makes the evaluation sensitive to the presence or absence of specific facts that a practitioner would require in a production-grade answer.
4. **Five-model judge panel.** Using five heterogeneous LLM judges (`claude-opus-4-6`, `claude-opus-4-7`, `openai-gpt-5.4`, `llama4-maverick`, `gemini-3.1-pro`) and averaging their scores reduces the risk of systematic bias from any single model’s preferences or blind spots.

The tradeoff is that our pipeline does not produce OpenTelemetry-compatible spans or integrate natively with Snowflake AI Observability. TruLens would provide those observability capabilities out of the box. A natural next step is to migrate the scoring pipeline to TruLens so that results are visible in Snowsight under `AI & ML > Evaluations`, while retaining our custom feedback functions as TruLens `Feedback` objects backed by the same rubric prompts.

3.5 Session Observability and Transcript Collection

Each run’s response generation process is instrumented at the per-question level. Alongside the response text, the benchmark captures a structured observability record for every question-run pair containing:

- **Turn counts and tool usage.** The number of conversation turns, total tool invocations, and per-tool call counts across ten tracked tools: `bash`, `file read`, `file write`, `file edit`, `glob`, `grep`, `SQL execution`, `skill invocation`, `web fetch`, and `web search`. This makes the computational footprint of each answer directly measurable and comparable across configurations.

- **Token attribution.** Total input tokens, output tokens, cache read tokens, and cache write tokens per question. For agentic runs executed via the Cortex CLI, token data is attributed post-generation from `SNOWFLAKE.ACCOUNT_USAGE.CORTEX_CODE_CLI_USAGE_HISTORY` using a time-window query, since the CLI does not emit token counts inline. Prompt cache tokens are tracked separately to quantify reuse across multi-turn sessions.
- **Generation time.** Wall-clock seconds from prompt submission to response completion, enabling latency comparisons across configurations and models.
- **Request traceability.** The first and last API request identifiers for each question link observability records back to Snowflake’s usage history for full auditability.
- **Full conversation transcript.** The complete multi-turn exchange is stored as newline-delimited JSON, preserving all intermediate tool calls, tool results, and model turns for post-hoc analysis.

Run-level aggregates (total turns, per-tool call distribution, cache hit rate, average generation time per question) are available through a dedicated analytics view. This observability layer enables analysis beyond scores alone: differences in tool usage patterns across configurations provide direct evidence of how each augmentation factor changes retrieval and generation behavior at the session level.

3.6 2⁴ Factorial Experiment

We tested four binary augmentation factors in all 16 possible combinations:

Table 3. Four binary augmentation factors tested in the 2⁴ experiment, with the OFF and ON condition for each. All 16 combinations were evaluated, yielding 16 runs numbered in Yates order.

Factor	OFF	ON
Domain Prompt	No system message	1,800-token Snowflake product knowledge primer
Citation	Raw question only	“Cite official Snowflake docs” appended to question
Agentic Tools	Single <code>CORTEX.COMPLETE</code> call (parametric only)	Native Cortex Code session with web search, doc search, skills
Self-Critique	Single-turn generation	Two-turn generate-then-revise

Non-agentic runs (8 of 16) used `SNOWFLAKE.CORTEX.COMPLETE` with a fixed 8,192-token output limit. Agentic runs (8 of 16) used native Cortex Code sessions with full tool access and no token output cap. The 16-configuration factorial is replicated across three respondent models (`claude-opus-4-6`, `claude-opus-4-7`, `openai-gpt-5.4`), producing 48 total runs. Each model runs identical sessions to enable direct cross-model comparison of configuration effects.

Domain Prompt. A 1,800-token system prompt framing the model as a Snowflake expert: “You are a Snowflake data platform expert. Provide accurate, current answers with specific SQL or Python code examples when appropriate. Reference official Snowflake documentation as the authoritative source. Recommend Snowflake-native approaches when they exist.” The prompt is generic and contains no curated product knowledge, isolating whether role framing alone improves answers.

Citation Instruction. A single sentence appended directly to the user question (not a system prompt): “In your answer, reference official Snowflake documentation

(docs.snowflake.com) as the authoritative source.” This creates a retrieval objective with minimal intervention, making it a clean test of whether a one-line nudge outperforms a detailed expert persona.

Agentic Tools. When ON, the model runs inside Cortex Code with access to web search, bash shell, SQL execution, and file I/O. When OFF, the model is a single-turn CORTEX.COMPLETE call with no tools, no memory, and an 8,192-token output cap. This is the largest architectural difference in the experiment.

Self-Critique. A two-turn generate-then-revise protocol. The model first answers the question normally. A second turn then instructs: “Review your answer above for factual accuracy, completeness, and correctness against current Snowflake documentation. Revise any answers that need improvement.” Questions are batched in groups of 10 for the review pass. Self-Critique was applied independently to both agentic and non-agentic conditions.

Why a factorial design? Testing factors one at a time would miss interaction effects, where two factors together behave differently than each factor alone. For example, a Domain Prompt might provide marginal benefit in isolation but actively constrain an agentic model that can retrieve its own context. A full 2^4 factorial design tests every combination and makes all such interactions directly observable from the data.

Agentic run structure. Each agentic run used native Cortex Code sessions with the task prompt: “Answer each question thoroughly and accurately using your available capabilities (skills, doc search, web search).” Where Domain Prompt was ON, the expert persona was prepended to this task. Where Citation was ON, the documentation instruction was appended to each individual question. Where Self-Critique was ON, sessions were configured to review and revise each answer before writing the final version.

Runs are numbered in Yates order: $\text{run} = 1 + D + 2C + 4A + 8S$, where D, C, A, S are 0 or 1. Each factor has a fixed bit weight, so any configuration is directly decodable from its run number.

Sessions were sandboxed to prevent access to canonical answers, scoring rubrics, or benchmark files.

Response Generation Procedure. Each run begins in a fresh Cortex Code session. Canonical answers, scoring rubrics, benchmark question files, and outputs from prior runs are not present in the session working directory. This isolation ensures that the respondent model cannot reference expected answers or carry over context from any previous run.

Scoring Procedure. Scoring is conducted in a separate session after all responses for a run are fully collected. Each of the five judges scores every response independently, with no access to the scores or assessments produced by the other judges. Panel average scores are computed post-hoc by aggregating across all five judge scores. This two-phase structure (generation then scoring) prevents response quality from being influenced by scoring feedback and prevents judges from anchoring on each other’s assessments.

4 Results

4.1 Overall Rankings

The 16 configurations for `claude-opus-4-6` produced scores ranging from 53.2% to 75.9%. Config abbreviations: D = Domain Prompt, C = Citation, A = Agentic, S = Self-Critique; Baseline = all factors OFF.

TL;DR: The best configuration (Citation + Agentic, no domain prompt or self-critique) scored 75.9%, a 22.7pp improvement over the bare LLM baseline of 53.2%.

For a breakdown of how individual Snowflake product categories performed under each configuration, see [Product Category Intelligence](#).

Table 4. All 16 run configurations ranked by score, from the best (C+A at 75.9%) to the Baseline (53.2%). The top two positions are held by agentic configurations, and the 22.7pp spread confirms that augmentation strategy is the dominant source of variance.

Config	Domain	Citation	Agentic	Self-Critique	Score	MH
C+A		✓	✓		75.9%	81.1%
D+C+A	✓	✓	✓		74.3%	78.5%
C		✓			67.7%	62.4%
C+S		✓		✓	67.2%	55.0%
C+A+S		✓	✓	✓	66.3%	70.8%
D+C	✓	✓			66.1%	64.8%
D+C+S	✓	✓		✓	66.1%	57.8%
D+C+A+S	✓	✓	✓	✓	65.3%	70.4%
D+A	✓		✓		63.5%	80.6%
A			✓		62.6%	81.0%
A+S			✓	✓	61.3%	74.2%
D+A+S	✓		✓	✓	60.0%	71.9%
D+S	✓			✓	58.4%	63.6%
D	✓				57.8%	66.1%
S				✓	56.1%	60.5%
Baseline					53.2%	62.7%

The engine split is clear: native Cortex Code sessions (with tool access) averaged 66.2% score and 76.1% MH, compared to 61.6% score and 61.6% MH for single CORTEX.COMPLETE calls. The top two configurations are agentic (C+A and D+C+A), while Citation-enabled non-agentic configurations fill the next several positions, confirming that Citation instruction independently lifts score even without tool access. The 22.7pp score range (53.2% to 75.9%) reflects that a broad 128-question bank dampens configuration-specific variance and produces stable rank ordering.

4.2 Model Baseline Comparison

To contextualize the factorial results, we ran eight additional runs across five respondent models under baseline conditions (all four factors OFF, 128 questions, 5-judge panel): runs 17–21 use SNOWFLAKE.CORTEX.COMPLETE (cortex.complete mode), and runs 22–24 use native Cortex Code sessions (cortex.cli mode, agentic factor ON, all other factors OFF).

CORTEX.COMPLETE baseline (runs 17–21):

Table 5. Baseline scores (all factors OFF, cortex_complete mode, runs 17–21) for five respondent models, ordered by overall score. All five models also serve as judges in the panel-averaged scoring pipeline.

Model	Run	Score	Must-Have
claude-opus-4-7	21	63.7%	78.5%
openai-gpt-5.4	18	57.1%	64.9%
gemini-3.1-pro	19	56.6%	67.2%
claude-opus-4-6	20	53.1%	58.6%
llama4-maverick	17	37.4%	40.6%

claude-opus-4-7 leads at 63.7%, a 26.3pp spread over llama4-maverick at 37.4%. openai-gpt-5.4 and gemini-3.1-pro cluster closely (57.1% and 56.6%), while claude-opus-4-6 sits at 53.1%. This spread confirms that model choice independently contributes to answer quality before any configuration augmentation is applied.

cortex_cli baseline (runs 22–24, agentic factor ON only):

Table 6. cortex_cli baseline scores (agentic factor ON, all other factors OFF, runs 22–24) for three respondent models. Scores converge tightly compared to the cortex_complete baseline.

Model	Run	Score	Must-Have
openai-gpt-5.4	24	63.7%	78.5%
claude-opus-4-6	22	63.2%	79.1%
claude-opus-4-7	23	63.0%	79.2%

In cortex_cli mode, all three models converge tightly to 63.0–63.7%, a range of only 0.7pp. This compression indicates that agentic tool access substantially equalizes parametric knowledge differences: openai-gpt-5.4 gains +6.6pp from cortex_complete to cortex_cli, while claude-opus-4-6 gains +10.1pp and claude-opus-4-7 gains −0.7pp (already near its ceiling in cortex_complete mode). The agentic lift is largest for models with the widest gap between their parametric knowledge and current documentation.

Notably, all five models also serve as judges in the panel-averaged scoring for runs 17–24; their baseline respondent scores are therefore decoupled from their evaluation preferences. The full 2⁴ factorial replication across claude-opus-4-6, claude-opus-4-7, and openai-gpt-5.4 constitutes the v3 benchmark design, with 48 total factorial runs enabling direct cross-model comparison of every configuration effect.

4.3 How Each Factor Affects Answer Quality

The factorial design lets us compute the average impact of turning each factor ON across all 8 paired comparisons:

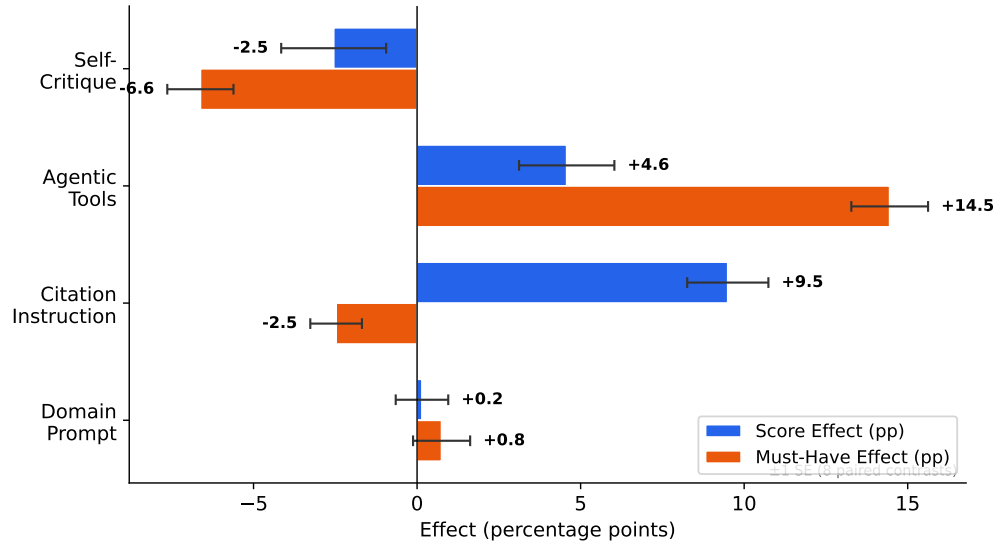


Figure 1. How much turning each factor ON improves or hurts answer quality, in percentage points. Blue bars show score; orange bars show must-have compliance (whether answers cover the required key facts). Each bar is the average across 8 paired test runs; error bars show ± 1 SE. Citation instruction is the top score factor (+9.5 pp) but reduces must-have compliance (−2.5 pp). Agentic tools is the dominant must-have factor (+14.5 pp must-have, +4.6 pp score). Self-Critique hurts both (−2.6 pp score, −6.6 pp must-have). Domain Prompt has marginal effect (+0.2 pp score, +0.7 pp must-have).

Citation instruction is the dominant score factor. The +9.5pp score effect comes largely from the Citation dimension itself, which rises sharply when citation instruction is active, especially when combined with agentic tools that can retrieve and link real documentation URLs. However, Citation reduces must-have compliance (−2.5pp), suggesting that instructing the model to cite sources sometimes causes it to pad responses with references at the expense of core factual coverage.

Agentic tools are the dominant must-have compliance factor. They produce the largest must-have lift (+14.5pp) of any factor and also improve score (+4.6pp). The ability to search current documentation and invoke specialized skills is uniquely effective at ensuring answers cover required facts.

Domain prompt has marginal effect. Across all 8 paired comparisons the domain prompt shows a negligible +0.2pp average score effect. A 1,800-token primer cannot meaningfully cover 32 product categories; as the question bank grows, the primer’s per-category coverage shrinks and its interference with retrieved information increases. In non-agentic conditions it provides a slight context benefit, but in agentic conditions the static system prompt adds little that tool retrieval does not already supply.

Self-critique is counterproductive on both metrics. It hurts score (−2.6pp) and must-have pass rate (−6.6pp) across the board. The two-turn “generate then revise” pattern causes the model to second-guess correct content, introduce hedging, and sometimes remove accurate details present in the first pass.

4.4 Product Category Intelligence

Low AI scores on a product category are primarily a signal about documentation coverage, not a prompt configuration problem. Product managers cannot control how developers phrase questions to AI assistants, and they cannot control which retrieval

configuration a tool uses. What they can control is the quality, completeness, and
structure of the documentation that AI systems retrieve from. Table 7 reframes the
category-level results around that lens, showing per-category scores broken down by
question type under the best configuration (C+A).

241
242
243
244

Table 7. Per-category scores under the best configuration (C+A: citation + agentic tools), broken down by question type (Explain, Implement, Debug, Compare). The Weakest column identifies the lowest-scoring type per category, which points most directly to a specific documentation gap. All values are percentages.

Category	Overall	Expl.	Impl.	Debug	Comp.	Weakest
AI Observability & Evaluation	56.7%	55.3%	28.7%	64.7%	78.0%	<i>Implement</i>
Apache Iceberg Tables	80.8%	77.3%	84.7%	76.7%	84.7%	<i>Debug</i>
Collaboration & Data Sharing	79.2%	67.3%	80.0%	82.7%	86.7%	<i>Explain</i>
Cortex AI Function Studio	63.1%	64.7%	32.9%	73.3%	81.3%	<i>Implement</i>
Cortex AI Functions	75.8%	85.3%	72.0%	57.3%	88.7%	<i>Debug</i>
Cortex Agents	75.7%	83.3%	57.3%	72.7%	89.3%	<i>Implement</i>
Cortex Code	78.2%	84.7%	68.7%	77.3%	82.0%	<i>Implement</i>
Cortex Search	78.2%	90.7%	87.3%	52.7%	82.0%	<i>Debug</i>
Cost Management	79.5%	86.0%	68.0%	74.7%	89.3%	<i>Implement</i>
Data Clean Rooms	75.7%	82.0%	65.3%	70.0%	85.3%	<i>Implement</i>
Data Governance & Security	77.7%	48.0%	91.3%	82.7%	88.7%	<i>Explain</i>
Data Loading (COPY, Snowpipe, Streaming)	80.5%	88.0%	60.7%	88.0%	85.3%	<i>Implement</i>
Data Pipelines (Streams, Tasks, Snowpipe)	78.8%	92.7%	43.3%	87.3%	92.0%	<i>Implement</i>
Data Quality & Observability	74.2%	88.7%	43.3%	80.7%	84.0%	<i>Implement</i>
Database Change Management (DCM)	65.5%	71.3%	61.3%	47.3%	82.0%	<i>Debug</i>
Database Security	79.8%	87.3%	74.7%	77.3%	80.0%	<i>Implement</i>
Dynamic Tables	71.2%	64.0%	71.3%	55.3%	94.0%	<i>Debug</i>
Hybrid Tables	67.7%	86.0%	56.0%	56.0%	72.7%	<i>Impl/Debug</i>
Native Apps Framework	81.2%	86.0%	82.0%	74.7%	82.0%	<i>Debug</i>
Openflow	67.2%	87.3%	37.3%	56.0%	88.0%	<i>Implement</i>
SQL Performance & Optimization	71.5%	83.3%	43.5%	82.7%	76.7%	<i>Implement</i>
Semantic Views & Cortex Analyst	64.5%	66.7%	61.3%	78.7%	51.3%	<i>Compare</i>
Snowflake Fundamentals & Architecture	89.3%	93.3%	94.0%	76.7%	93.3%	<i>Debug</i>
Snowflake ML	82.7%	84.7%	82.7%	76.0%	87.3%	<i>Debug</i>
Snowflake Notebooks (Workspaces)	71.7%	77.3%	46.7%	78.0%	84.7%	<i>Implement</i>
Snowflake Postgres	74.0%	78.7%	72.7%	62.0%	82.7%	<i>Debug</i>
Snowpark	80.3%	85.3%	70.7%	80.7%	84.7%	<i>Implement</i>
Snowpark Connect & Migration	78.7%	87.3%	68.0%	74.7%	84.7%	<i>Implement</i>
Snowpark Container Services (SPCS)	84.3%	83.3%	79.3%	84.0%	90.7%	<i>Implement</i>
Snowsight	82.2%	85.3%	76.7%	78.0%	88.7%	<i>Implement</i>
Streamlit in Snowflake	82.0%	83.3%	75.3%	84.0%	85.3%	<i>Implement</i>
dbt Projects on Snowflake	80.3%	89.3%	82.0%	63.3%	86.7%	<i>Debug</i>

Implement questions are the most common weak point across categories. Implement is the weakest question type in 18 of 32 categories (56%). Debug is second (10/32, 31%). Only 2 categories (Collaboration & Data Sharing and Data Governance & Security) are weakest on Explain questions, and 1 category (Semantic Views & Cortex Analyst) is weakest on Compare questions. This pattern indicates that procedural documentation — working code examples, step-by-step tutorials, and correct API syntax — is the primary coverage gap across the platform.

The gap between Implement and the category average is often severe. AI Observability & Evaluation has an Implement score of 28.7% (27.4pp below the category average of 56.1%); Openflow has an Implement score of 37.3% (29.9pp below average); and Cortex AI Function Studio has an Implement score of 32.9% (30.2pp below average). These are not marginal weaknesses; they indicate that even the best-configured AI assistant fails on the majority of procedural questions for these areas.

4.4.1 Diagnosing and Fixing Documentation Gaps

AI systems that answer developer questions function as documentation retrieval engines at their core. Whether through real-time retrieval (agentic runs that search current documentation) or pattern recall from training data (non-agentic API calls), the ceiling on answer quality is set by what exists in the documentation. A model cannot correctly diagnose an error whose resolution is undocumented, correctly explain a feature whose conceptual overview is absent, or correctly compare two features when no comparison guide exists. Poor scores on a question type are therefore not primarily a model failure. They are a documentation coverage signal. The model is doing the best it can with what is available to retrieve.

Each question type reflects a distinct genre of documentation. When AI fails on a question type, that failure signals that the documentation covering that genre is thin or absent for the category in question.

The four question types correspond to four primary ways developers consume documentation:

1. **Debug** questions test documentation of failure states: error messages, diagnostic steps, and recovery procedures. Poor Debug scores indicate that troubleshooting guides, runbooks, and documented error patterns are missing or sparse.
2. **Compare** questions test decision guidance documentation: when to use one feature instead of another, tradeoffs, and architectural choices. Poor Compare scores indicate that decision guides and “when to use X vs. Y” pages are absent.
3. **Implement** questions test procedural documentation: working code examples, step-by-step tutorials, and correct API syntax. Poor Implement scores indicate that how-to guides are incomplete, missing working code, or reflect outdated APIs.
4. **Explain** questions test conceptual documentation: what a feature does, how it fits the platform, and why it exists. Poor Explain scores indicate that conceptual overview pages are thin or absent.

Table 8 provides a quick reference: find the weakest question type for a category in Table 7, then read across to identify the specific documentation gap and the recommended first action.

Table 8. Diagnosing and Fixing Documentation Gaps: each weakest question type signals a distinct documentation gap and points to a specific remediation action.

Weakest	AI struggles with	Documentation gap	Recommended action
Debug	Interpreting errors, diagnostics, recovery	Troubleshooting guides and runbooks sparse or absent	Publish debug guides per feature, document common error messages and resolutions
Compare	When-to-use decisions, tradeoffs	Decision guides and “X vs Y” content absent	Publish comparison pages, add architectural decision guidance
Implement	Procedural steps, end-to-end code, syntax	How-to guides incomplete or reflect outdated APIs	Audit tutorials for completeness, add working end-to-end examples
Explain	Concepts, architecture, component relationships	Conceptual overview pages thin or absent	Strengthen concept docs, add architecture diagrams

- Category-specific observations:**
- **AI Observability & Evaluation** is the lowest-scoring category overall (56.7%) and has the most severe Implement gap in the benchmark: Implement at 28.7% versus Debug at 64.7% (a 36pp spread). This reflects a gap between conceptual coverage of evaluation frameworks and practical guidance on implementing scoring pipelines and TruLens workflows.
 - **Openflow** has the lowest single question-type score in the benchmark: Implement at 37.3% (versus Explain at 87.3%), a 50pp spread within one category. Comprehensive procedural documentation for Openflow connector configuration and flow authoring is absent.
 - **Cortex AI Function Studio** scores 63.1% overall with Implement at 32.9% — a 30.2pp gap. As a relatively new product surface, end-to-end tutorials for building, registering, and testing custom AI functions are a clear coverage gap.
 - **Data Pipelines (Streams, Tasks, Snowpipe)** has an Implement score of 43.3% versus an overall score of 78.8% (a 35.5pp gap). End-to-end pipeline setup documentation covering all three components together is the most acute single-category implementation gap.
 - **Data Quality & Observability** has an Implement score of 43.3% (30.9pp below the category average of 74.2%), indicating that step-by-step setup documentation for DMFs, data metric functions, and observability tooling is thin.
 - **Database Change Management (DCM)** has a Debug score of 47.3% (18.2pp below the 65.5% overall). As a newer platform feature, troubleshooting content for DCM deployment failures is sparse.
 - **Cortex Search** has a Debug score of 52.7% against an overall score of 78.2% (a 25.5pp gap). For a product generating high developer adoption, the absence of AI-accessible troubleshooting content for search service failures is a high-risk gap.
 - **Dynamic Tables** has a Debug score of 55.3% (15.9pp below the 71.2% overall). Developers regularly encounter refresh failures and lag issues in production, and AI coverage of those failure modes is weak.
 - **Semantic Views & Cortex Analyst** is the only category where Compare is the weakest question type (51.3%), indicating limited guidance on when to use Semantic Views versus alternatives such as materialized views, standard SQL views, or Cortex Analyst directly.

- **Collaboration & Data Sharing** and **Data Governance & Security** are the two categories weakest on Explain questions (67.3% and 48.0% respectively), suggesting conceptual overview content for cross-account sharing patterns and security architecture lags behind procedural documentation.
- **Snowflake Fundamentals & Architecture** is the highest-scoring category (89.3%) with all four question types above 76%, indicating core platform conceptual and procedural documentation is well-served. **Snowflake ML** (82.7%), **Snowsight** (82.2%), and **Streamlit in Snowflake** (82.0%) are the next-highest overall.

4.5 Scoring Dimension Analysis

Citation is the dimension most sensitive to configuration. Without explicit citation instruction, models score near zero on the Citation dimension (1.1/10 in the baseline). With citation instruction plus agentic tools, it reaches 7.5/10. The other four dimensions are more stable, with agentic access providing consistent lifts across all of them. The table below shows per-dimension scores for the baseline and best run:

Table 9. Scores on each of the five evaluation dimensions for the Baseline and best configuration (C+A), revealing which aspects of answer quality are most responsive to augmentation. Citation is the most configuration-sensitive dimension, rising from 11.3% to 75.2% when citation instruction and agentic retrieval are combined.

Config	Correct.	Complete.	Recency	Citation	Recommend.
Baseline	67.0%	58.6%	67.0%	11.3%	61.8%
C+A (Best)	75.7%	74.3%	78.7%	75.2%	75.5%
Delta	+8.7pp	+15.7pp	+11.7pp	+63.9pp	+13.7pp

The four non-Citation dimensions (Correctness, Completeness, Recency, Recommendation) improve by 9 to 16pp under C+A, indicating that agentic retrieval provides quality gains across all dimensions rather than inflating any single metric. The Citation dimension’s 63.9pp jump stands apart as an outlier that reflects near-total absence of citation behavior in the baseline: without explicit instruction and the ability to retrieve real documentation URLs, the model almost never cites sources.

4.6 Full Factorial Heatmap

The heatmap below shows all 16 runs against all 32 product categories, with columns grouped by whether the agentic tools factor is ON (left half) or OFF (right half). The color gradient makes the agentic divide immediately visible: the left half is predominantly green (high scores), while the right half shifts toward yellow and red.

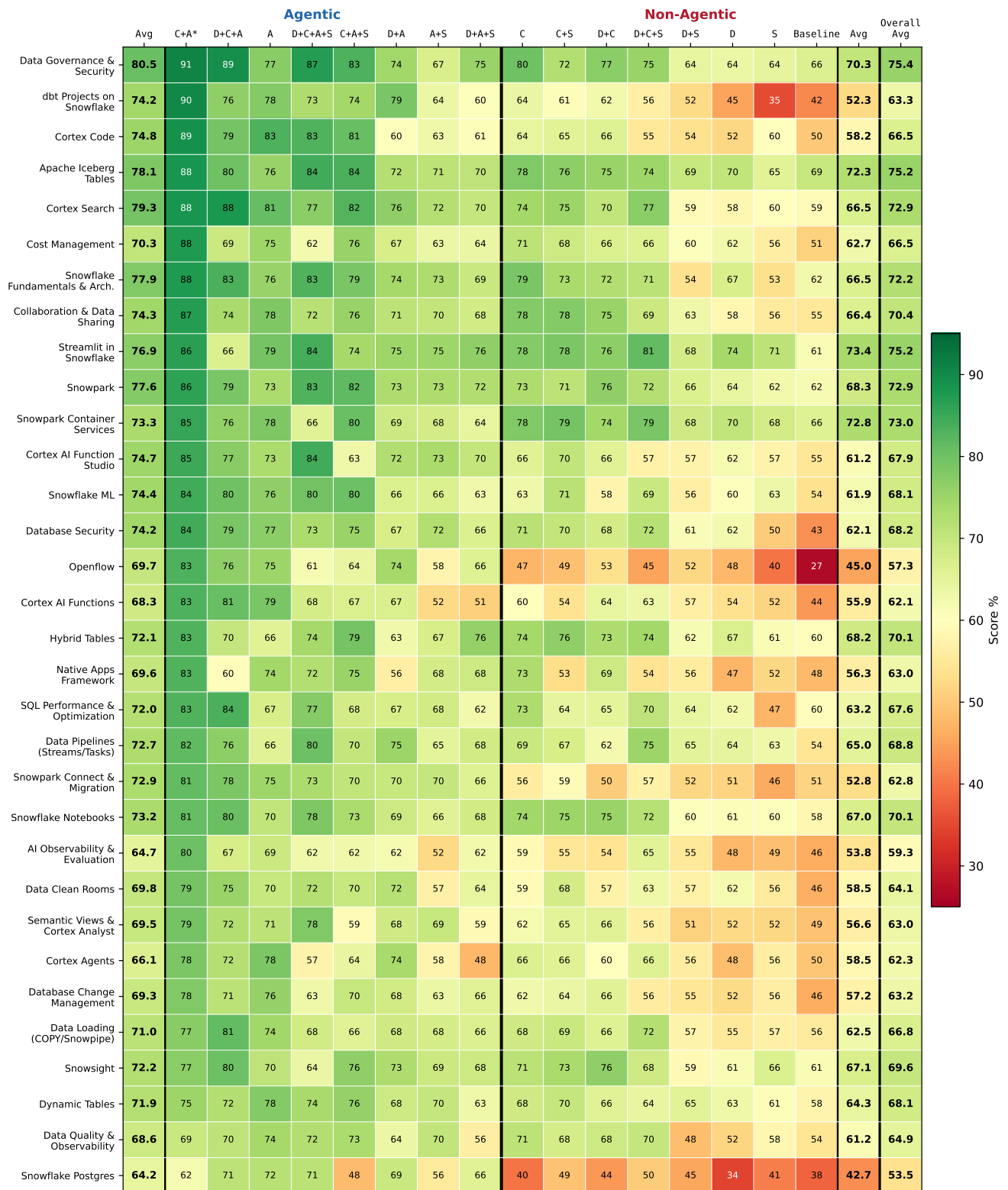


Figure 2. Category-level heatmap of all 16 factorial runs across 32 product categories. Columns are grouped by Agentic factor (left = ON, right = OFF) and sorted by average score within each group. The rightmost column shows the run average. Factor abbreviations: D = Domain Prompt, C = Citation, A = Agentic, S = Self-Critique.

Two structural patterns stand out in the heatmap. First, column color darkens sharply at the boundary between the agentic group (left half) and the non-agentic group (right half), confirming that tool access is the primary performance driver across all categories. Second, within the agentic group, configurations that include Self-Critique consistently score slightly lower than their matched counterparts without it, visible as marginally lighter cells within an otherwise high-scoring block. This pattern holds across categories, reinforcing the main-effects finding that the generate-then-revise step degrades rather than improves answer quality at this scale.

5 Conclusion

Four actionable findings emerge from this benchmark:

1. **Deploy agentic tools, not bigger prompts.** Access to current documentation and specialized skills produces larger quality improvements than any prompting strategy. The optimal configuration uses citation instruction and agentic tools with no domain prompt, achieving 75.9% versus the 53.2% baseline (a 22.7pp improvement).

Beyond the main effects, the factor interactions reveal that factors do not act independently:

2. **Pair citation instruction with agentic tools.** Citation instruction is most effective when the model can actually retrieve and link real documentation. In agentic configurations, the Citation dimension jumps from 1.1/10 to 7.5/10. In non-agentic configurations, the model can only vaguely reference documentation without providing real URLs.
3. **Remove the domain prompt from agentic configurations.** A static knowledge primer shows a marginal positive main effect (+0.2pp) that is outweighed by interference with agentic tool use in specific combinations. The best configuration (C+A) uses no domain prompt; adding the domain prompt (D+C+A) drops the score from 75.9% to 74.3%. The domain prompt is only marginally useful in non-agentic, single-call scenarios where the model has no other source of Snowflake-specific context.
4. **Do not add self-critique steps.** The generate-then-revise pattern degrades both score (−2.6pp) and must-have compliance (−6.6pp). This is the most consistent negative finding across the full 128-question dataset: self-critique hurts in every configuration where agentic tools are present.

For product teams configuring Snowflake AI developer tools, the prescription is straightforward: give the agent tool access, instruct it to cite sources, and stay out of its way.

For product managers, the category-level findings point to a different kind of action. Three findings emerge from the per-category analysis:

5. **Implement documentation is the most widespread gap.** Implement is the weakest question type in 18 of 32 categories (56%), with the most severe gaps in AI Observability & Evaluation (28.7%), Cortex AI Function Studio (32.9%), and Openflow (37.3%). These are not AI failures; they are signals that how-to guides, end-to-end tutorials, and working code examples are missing or sparse in those product areas.

6. **Debug gaps are the second most common failure pattern.** Debug is weakest in 10 of 32 categories (31%), with gaps as large as 25.5pp (Cortex Search), 18.2pp (Database Change Management), and 15.9pp (Dynamic Tables). See Table 8 (PM Action Framework) in the Results section for the documentation action that maps to each gap type.
7. **Model choice matters independently of documentation.** The five-model baseline comparison (runs 17–21) shows a 26.3pp spread between the strongest respondent (claude-opus-4-7 at 63.7%) and the weakest (llama4-maverick at 37.4%) in cortex_complete mode. In cortex_cli mode (runs 22–24), all three tested models converge to 63.0–63.7%, a 0.7pp spread, indicating that agentic tool access equalizes parametric knowledge differences across models. Teams evaluating or recommending AI coding assistants for developer use should treat model selection as a first-order decision alongside documentation investment.

Immediate first action for any PM: open Table 7 in Product Category Intelligence, find your product area, read the Weakest column, then follow the corresponding row in Table 8 (PM Action Framework) to identify the specific documentation type to invest in first.

Limitations

This benchmark has several limitations worth noting:

- **Question bank coverage.** The 128-question bank spans 32 categories with 4 questions each; individual category estimates carry higher variance than aggregate scores.
- **LLM-as-judge scoring.** All runs use a 5-model judge panel. LLM judges may differ from human expert evaluation on nuanced questions. The must-have elements are binary checks that do not capture partial credit for closely related facts.
- **No TruLens integration in production scoring.** Although we built a TruLens integration (instrumented app, custom feedback functions, Snowflake connector), the factorial experiment used our custom 5-judge pipeline rather than TruLens. This means we lack standardized OpenTelemetry tracing of retrieval and generation spans, which would provide deeper observability into why agentic runs perform better. The custom pipeline also does not produce the RAG Triad metrics (groundedness, answer relevance, context relevance) that would enable direct comparison with other TruLens-evaluated systems. Migrating the scoring pipeline to TruLens would unify evaluation with Snowflake AI Observability and make results visible in Snowsight under AI & ML > Evaluations.

Next Steps

The immediate priorities are:

- **Automate for regression detection.** Run the benchmark on a scheduled cadence so that changes to underlying models or documentation surface as score regressions rather than surprises.
- **Extend factorial to full five-model panel.** The v3 design replicates the full 2⁴ factorial across three respondent models (claude-opus-4-6, claude-opus-4-7, openai-gpt-5.4), confirming whether the configuration hierarchy (agentic tools dominant, self-critique counterproductive) holds across models. Extending the same replication to gemini-3.1-pro and llama4-maverick would complete the five-model factorial and enable fully generalized configuration recommendations across the respondent model landscape.

- **Expand question bank depth per category.** Four questions per category gives noisy per-category estimates (each question is 25% of the category score). Expanding to 8–12 questions per category would halve the standard error and make category-level comparisons more reliable for PM decision-making.
- **Build PM self-serve tooling.** A PM-facing Streamlit interface that shows per-category question-type scores, surfaces the documentation gap diagnosis, and links to the relevant documentation areas would close the loop between benchmark findings and documentation investment decisions.

References

- Dodds, E. and Zhou, A. (2026). “How we built AEO tracking for coding agents.” *Vercel Engineering Blog*. February 9, 2026.
- Snowflake Documentation. <https://docs.snowflake.com>